Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по проектной работе по дисциплине

«Математический анализ»

«Математический анализ в методах машинного обучения»

Выполнили: Вейгандт В.
Зайцев Н.
Горобец О.
Козлов В.
Шагеев Э.
Группа: Р 1.2

Преподаватель: Карпычев А.В.

Санкт-Петербург
2026

Содержание

Введение **3**

1 Постановка задачи машинного обучения **4**
1.1 Объекты, признаки и выборка 4
1.2 Модель и алгоритм обучения 4
1.3 Функция потерь и истинный риск 5
1.4 Эмпирический риск и обобщающая способность 5

2 Линейные модели **6**
2.1 Линейная регрессия и её параметры 6
2.2 Остатки и среднеквадратичная ошибка 6
2.3 MSE как функция нескольких переменных 6
2.4 Градиент MSE 7
2.5 Логистическая регрессия и сигмоида 7
2.6 Бернулли, правдоподобие и Binary Cross-Entropy 8
2.7 Градиент бинарной кросс-энтропии 8

3 Градиентные методы оптимизации **8**
3.1 Градиент и производная по направлению 8
3.2 Gradient Descent: метод и локальная аппроксимация 9
3.3 Batch, Stochastic и Mini-batch Gradient Descent 10
3.4 Momentum и Nesterov Momentum 10
3.5 AdaGrad и RMSProp 10
3.6 Adam 11

4 Решающие деревья и ансамбли **11**
4.1 Структура решающего дерева 11
4.2 Критерий качества разбиения 12
4.3 Жадное построение и дискретная оптимизация 12
4.4 Random Forest: ансамбль параллельных деревьев 13
4.5 Gradient Boosting как последовательное улучшение функций 13
4.6 Связь ансамблей с дальнейшим изложением 14

5 Нейронные сети **14**
5.1 Искусственный нейрон и функции активации 14
5.2 Многослойный перцептрон как композиция функций 15
5.3 Почему скрытые слои необходимы: задача XOR 15
5.4 Выход классификационной сети: scores и Softmax 16
5.5 Градиент Softmax и кросс-энтропии 16
5.6 Обучение нейросети и роль Backpropagation 17
5.7 Правило цепочки и матрица Якоби 17
5.8 Обратное распространение по слоям 18
5.9 Затухание градиента и остаточные связи ResNet 18
5.10 Свёрточные нейронные сети 19

6 Муравьиный алгоритм: пример оптимизации без градиентов 20
6.1 Задача коммивояжёра 20
6.2 Вероятностный выбор перехода 20
6.3 Испарение и обновление феромона 21
6.4 Полный алгоритм и ограничения метода 21

7 Распределение работы между участниками 21

Заключение 22

Список источников 23

А Материалы проекта 23

Введение

Машинное обучение (Machine Learning, ML) — область информатики и прикладной математики, изучающая построение моделей, закономерности работы которых определяются наблюдаемыми данными. Вместо явного описания всех правил решения задачи задаётся семейство допустимых моделей и процедура выбора подходящей модели по обучающей выборке.

С математической точки зрения обучение параметрической модели часто является задачей минимизации функции многих переменных. Здесь используются понятия градиента, частной производной, производной композиции, экстремума, функции потерь и итерационного процесса. Однако не все рассматриваемые методы оптимизируются одинаково: структура решающего дерева выбирается дискретно, градиентный бустинг последовательно строит сумму функций, а нейронная сеть представляет собой композицию обучаемых преобразований.

Цель работы — последовательно изложить основные математические конструкции, лежащие в основе ряда методов машинного обучения, и показать, как они связаны с курсом математического анализа. Работа носит *теоретико-обзорный* характер и служит основой для лекционного доклада.

Для достижения цели поставлены следующие задачи:

1) формализовать обучение с учителем и принцип минимизации эмпирического риска;

2) разобрать линейную и логистическую регрессии, их функции потерь и градиенты;

3) изложить геометрический смысл градиентного спуска и основные модификации метода;

4) объяснить жадное построение решающих деревьев, случайный лес и градиентный бустинг;

5) описать многослойные нейронные сети, обучение с помощью обратного распространения ошибки, а также идеи CNN и ResNet;

6) рассмотреть муравьиный алгоритм как пример вероятностной оптимизации в дискретном пространстве решений.

Изложение следует логике лекционного материала: *данные — модель — функция потерь — способ обучения*. Все основные обозначения вводятся при первом использовании, а математические результаты приводятся с пояснениями, необходимыми для понимания метода. Теоретической основой служат лекции ШАД, учебные материалы Яндекса, *Dive into Deep Learning* и другие источники, приведённые в библиографии [1, 2, 3, 4, 6].

1. Постановка задачи машинного обучения

1.1. Объекты, признаки и выборка

Пусть $\mathcal{X}$ — множество объектов, $\mathcal{Y}$ — множество истинных ответов, а (X, Y) — случайная пара с неизвестным совместным распределением $P_{X,Y}$. Наблюдается выборка

$$D_n = \{(x_i, y_i)\}_{i=1}^n, \qquad (X_i, Y_i) \overset{\text{iid}}{\sim} P_{X,Y}. \tag{1}$$

Запись iid (independent and identically distributed) означает, что случайные пары независимы и имеют одно распределение; (x_i, y_i) обозначает полученные значения (X_i, Y_i). Предположение о независимости и одинаковом распределении удобно для теоретической постановки, но в реальных данных выполняется не всегда.

Если объект описывается d числовыми признаками, то

$$x_i = (x_{i1}, \ldots, x_{id})^T \in \mathbb{R}^d, \qquad X = \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} \in \mathbb{R}^{n \times d}.$$

Индекс i нумерует объекты, а индекс j — признаки. Матрицу X называют матрицей признаков; вектор ответов $y = (y_1, \ldots, y_n)^T$ имеет длину n.

Различают две основные постановки:

- *регрессия* — предсказание вещественной величины ($\mathcal{Y} = \mathbb{R}$), например цены квартиры;
- *классификация* — определение класса ($\mathcal{Y} = \{1, \ldots, K\}$), например категории изображения.

Важно отличать истинный ответ $y \in \mathcal{Y}$ от результата модели $\hat{y} \in \widehat{\mathcal{Y}}$. При классификации модель может возвращать не метку класса, а вероятности принадлежности к классам. Тогда пространство прогнозов $\widehat{\mathcal{Y}}$ не совпадает с $\mathcal{Y}$.

1.2. Модель и алгоритм обучения

Определение. Параметрической моделью называется семейство функций

$$\mathcal{F} = \{f_\theta : \mathcal{X} \to \widehat{\mathcal{Y}} \mid \theta \in \Theta\}, \tag{2}$$

где Θ — допустимое множество параметров. Например, семейство $f_{w,b}(x) = wx + b$ содержит все прямые, задаваемые коэффициентами $w, b \in \mathbb{R}$.

Алгоритм обучения — правило выбора параметров по данным:

$$A : D_n \mapsto \hat{\theta}, \qquad \hat{\theta} = A(D_n).$$

Таким образом, модель определяет *какие функции допустимы*, а алгоритм обучения — *как выбрать конкретную функцию*. После обучения используется модель $f_{\hat{\theta}}$.

1.3. Функция потерь и истинный риск

Для количественного сравнения предсказания и правильного ответа задаётся *функция потерь* (loss function)

$$\ell : \mathcal{Y} \times \widehat{\mathcal{Y}} \to \mathbb{R}_{\geq 0}.$$

Например, для регрессии подходит квадратичная потеря $\ell(y, \hat{y}) = (y - \hat{y})^2$. При классификации с предсказанной *меткой* класса может использоваться индикатор ошибочного ответа

$$\ell_{0/1}(y, \hat{y}) = \mathbf{1}\{y \neq \hat{y}\},$$

где $\mathbf{1}\{A\}$ равно единице, если условие A выполнено, и нулю иначе. Если же модель выдаёт вероятности классов, применяются другие потери, например кросс-энтропия.

Определение. Истинный риск — математическое ожидание потери на новой паре (X, Y) из того же распределения:

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P_{X,Y}} \left[\ell\big(Y, f_\theta(X)\big) \right]. \tag{3}$$

Математическое ожидание означает усреднение по возможным объектам и ответам с учётом их вероятностей. Желательно получить малое значение $R(\theta)$, однако распределение $P_{X,Y}$ обычно неизвестно.

1.4. Эмпирический риск и обобщающая способность

Наблюдаемые данные позволяют вычислить среднюю ошибку на выборке:

$$\widehat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(y_i, f_\theta(x_i)\big). \tag{4}$$

Принцип ERM (Empirical Risk Minimization, минимизация эмпирического риска) заключается в выборе

$$\hat{\theta} \in \arg\min_{\theta \in \Theta} \widehat{R}_n(\theta). \tag{5}$$

Символ $\arg\min$ обозначает *аргументы*, то есть значения параметра, на которых достигается минимум функции, а не само минимальное значение. Поскольку минимум может достигаться на нескольких наборах параметров, используется знак $\in$.

При фиксированном θ и существовании конечного математического ожидания закон больших чисел даёт сходимость выборочного среднего $\widehat{R}_n(\theta)$ к $R(\theta)$ при $n \to \infty$. *Это не означает автоматически*, что минимизатор эмпирического риска $\hat{\theta}$, зависящий от выборки, хорошо обобщает данные: для таких выводов требуются дополнительные условия.

Обобщающая способность (generalization) — сохранение качества предсказаний на новых объектах. *Переобучение* (overfitting) обычно проявляется низкой ошибкой на обучении и существенно большей на независимой тестовой выборке. *Недообучение* (underfitting) соответствует недостаточной способности модели описать закономерность даже обучающих данных.

При практической оценке качества различают обучающую выборку D_{train} (подбор параметров), валидационную D_{val} (выбор настроек и варианта модели) и тестовую D_{test} (независимая итоговая оценка). Тестовая выборка не должна использоваться при подборе модели; иначе оценка её качества будет смещена оптимистически.

2. Линейные модели

2.1. Линейная регрессия и её параметры

Линейная регрессия (Linear Regression) используется для прогнозирования вещественного ответа. Её простейшая модель имеет вид

$$f_{w,b}(x) = w^T x + b, \qquad w \in \mathbb{R}^d, \quad b \in \mathbb{R}. \tag{6}$$

Здесь w — вектор весов, b — свободный член, а $w^T x = \sum_{j=1}^{d} w_j x_j$ — скалярное произведение.

Для сокращения записи вводят расширенный вектор признаков $\widetilde{x} = (1, x_1, \ldots, x_d)^T$ и вектор параметров $\theta = (b, w_1, \ldots, w_d)^T$. Тогда

$$f_\theta(x) = \widetilde{x}^T \theta, \qquad \widetilde{X} = \begin{pmatrix} \widetilde{x}_1^T \\ \vdots \\ \widetilde{x}_n^T \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

Это обозначение устраняет необходимость выписывать свободный член отдельно. В дальнейшем X обозначает исходную матрицу признаков, а $\widetilde{X}$ — матрицу с добавленным столбцом единиц. Линейность регрессионной модели понимается прежде всего *по параметрам*: модель $\theta_0 + \theta_1 x + \theta_2 x^2$ тоже линейна по θ, хотя не является линейной функцией x.

2.2. Остатки и среднеквадратичная ошибка

Пусть $\widehat{y}_i = \widetilde{x}_i^T \theta$ — предсказание, а $r_i = y_i - \widehat{y}_i$ — остаток (residual). Для всей выборки

$$\widehat{y} = \widetilde{X}\theta, \qquad r = y - \widetilde{X}\theta.$$

Определение. Среднеквадратичная ошибка (Mean Squared Error, MSE):

$$L_{\text{MSE}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \widetilde{x}_i^T \theta)^2 = \frac{1}{n} \|y - \widetilde{X}\theta\|_2^2. \tag{7}$$

Здесь $\|v\|_2 = \sqrt{v_1^2 + \cdots + v_n^2}$ — *евклидова норма* (длина вектора), а $\|v\|_2^2 = \sum_{i=1}^{n} v_i^2$.

Например, если истинные ответы равны $(2, 4, 5)$, а предсказания — $(3, 3, 5)$, остатки составляют $(-1, 1, 0)$, а MSE равна $(1 + 1 + 0)/3 = 2/3$. В отличие от суммы абсолютных ошибок квадратичная потеря сильнее штрафует большие отклонения и удобна для дифференцирования.

2.3. MSE как функция нескольких переменных

При одном исходном признаке и двух параметрах w, b получаем

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - w x_i - b)^2, \qquad L : \mathbb{R}^2 \to \mathbb{R}.$$

Поэтому регрессия непосредственно связана с изучением функций нескольких переменных. *Линия уровня* $L(w, b) = c$ объединяет все пары параметров с одинаковым значением ошибки.

MSE представляет собой выпуклую квадратичную функцию параметров. Это означает, что для любых θ_1, θ_2 и $\lambda \in [0, 1]$

$$L\big(\lambda\theta_1 + (1-\lambda)\theta_2\big) \le \lambda L(\theta_1) + (1-\lambda)L(\theta_2).$$

Следовательно, любой её локальный минимум является глобальным. В зависимости от свойств данных минимум может быть не единственным.

2.4. Градиент MSE

При $d = 1$ частные производные выражаются через остатки:

$$\frac{\partial L}{\partial w} = -\frac{2}{n}\sum_{i=1}^{n} x_i(y_i - wx_i - b), \qquad \frac{\partial L}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - wx_i - b).$$

В общем случае дифференцирование каждого слагаемого по компоненте θ_j с использованием правила цепочки даёт

$$\frac{\partial L}{\partial \theta_j} = \frac{2}{n}\sum_{i=1}^{n} \widetilde{x}_{ij}(\widetilde{x}_i^T\theta - y_i).$$

Собирая частные производные в вектор, получаем

$$\boxed{\nabla_\theta L_{\text{MSE}}(\theta) = \frac{2}{n}\widetilde{X}^T(\widetilde{X}\theta - y)}. \tag{8}$$

Обучение регрессии заключается в поиске параметров, минимизирующих (7). Выражение (8) позволяет выполнять итерационные шаги уменьшения функции потерь; соответствующие методы рассматриваются в следующей главе.

2.5. Логистическая регрессия и сигмоида

Перейдём к бинарной классификации $y \in \{0, 1\}$. Линейное выражение $z = \widetilde{x}^T\theta$ может принимать любое вещественное значение и поэтому само по себе не является вероятностью. В логистической регрессии моделируют условную вероятность класса 1:

$$p_\theta(x) = P_\theta(Y = 1 \mid X = x).$$

Предполагается линейная зависимость *логарифма отношения шансов* (log-odds) от признаков:

$$\log\frac{p_\theta(x)}{1 - p_\theta(x)} = \widetilde{x}^T\theta = z.$$

Отсюда $p_\theta(x)/(1 - p_\theta(x)) = e^z$, значит

$$p_\theta(x) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z).$$

Функция σ называется *сигмоидой*; её значения находятся в интервале $(0, 1)$, при этом $\sigma(0) = 1/2$ и

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big).$$

При пороге $1/2$ модель относит объект к классу 1, когда $\widetilde{x}^T\theta \ge 0$. Следовательно, граница принятия решения задаётся гиперплоскостью $\widetilde{x}^T\theta = 0$. Несмотря на нелинейную сигмоиду, граница классификации остаётся линейной по признакам.

2.6. Бернулли, правдоподобие и Binary Cross-Entropy

Пусть $p_i = p_\theta(x_i)$. Для бинарного ответа используем модель Бернулли:

$$P_\theta(Y_i = y_i \mid X_i = x_i) = p_i^{y_i}(1-p_i)^{1-y_i}.$$

При условной независимости ответов правдоподобие выборки равно

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i}. \tag{9}$$

Правдоподобие — вероятность наблюдаемых ответов при заданных входных признаках, рассматриваемая как функция параметров модели. *Метод максимального правдоподобия* (Maximum Likelihood Estimation, MLE) выбирает параметры, увеличивающие (9).

Поскольку логарифм строго возрастает, максимизация $\mathcal{L}(\theta)$ эквивалентна минимизации отрицательного среднего логарифма правдоподобия:

$$\boxed{L_{\text{BCE}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]}. \tag{10}$$

Эта функция называется *бинарной кросс-энтропией* (Binary Cross-Entropy, BCE).

2.7. Градиент бинарной кросс-энтропии

Для одного наблюдения $\ell_i = -y_i \log p_i - (1-y_i)\log(1-p_i)$, $p_i = \sigma(z_i)$. Дифференцируя последовательно по p_i и z_i, получаем

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}, \qquad \frac{dp_i}{dz_i} = p_i(1-p_i).$$

После сокращения множителей

$$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i.$$

Так как $z_i = \widetilde{x}_i^T \theta$, правило цепочки даёт

$$\boxed{\nabla_\theta L_{\text{BCE}}(\theta) = \frac{1}{n}\widetilde{X}^T(p-y)}, \qquad p = (p_1, \ldots, p_n)^T. \tag{11}$$

Сравнение (8) и (11) показывает общую структуру: вектор отклонений предсказаний от ответов умножается на транспонированную матрицу признаков. Это естественный переход к градиентным методам обучения.

3. Градиентные методы оптимизации

3.1. Градиент и производная по направлению

Пусть $L : \mathbb{R}^m \to \mathbb{R}$ — дифференцируемая функция параметров $\theta = (\theta_1, \ldots, \theta_m)^T$. *Градиентом* называется вектор частных производных

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_m}\right)^T.$$

Для векторов $u, v \in \mathbb{R}^m$ введём скалярное произведение $\langle u, v\rangle = \sum_{j=1}^m u_j v_j$ и евклидову норму $\|v\|_2 = \sqrt{\langle v, v\rangle}$. Если v — единичный вектор, то *производная по направлению* характеризует локальное изменение L при смещении вдоль v:

$$D_v L(\theta) = \lim_{h\to 0} \frac{L(\theta + hv) - L(\theta)}{h} = \langle \nabla L(\theta), v\rangle.$$

Согласно неравенству Коши–Буняковского,

$$-\|\nabla L(\theta)\|_2 \le D_v L(\theta) \le \|\nabla L(\theta)\|_2.$$

Если $\nabla L(\theta) \neq 0$, минимальное значение производной по единичному направлению достигается при

$$v = -\frac{\nabla L(\theta)}{\|\nabla L(\theta)\|_2}.$$

Таким образом, *антиградиент* задаёт направление наиболее быстрого *локального* убывания функции в евклидовой геометрии. Это утверждение само по себе не гарантирует достижения глобального минимума.

3.2. Gradient Descent: метод и локальная аппроксимация

При малом $h \in \mathbb{R}^m$ дифференцируемость даёт приближение

$$L(\theta + h) = L(\theta) + \langle \nabla L(\theta), h\rangle + o(\|h\|_2). \tag{12}$$

Здесь $o(\|h\|_2)$ означает остаток, отношение которого к $\|h\|_2$ стремится к нулю при $h \to 0$. Подставляя $h = -\eta \nabla L(\theta)$, где $\eta > 0$ — размер шага (learning rate), получаем локальную оценку

$$L(\theta - \eta \nabla L(\theta)) \approx L(\theta) - \eta \|\nabla L(\theta)\|_2^2.$$

При достаточно малом шаге ненулевой градиент позволяет уменьшить значение функции.

Определение. Градиентный спуск (Gradient Descent, GD) строит последовательность параметров по правилу

$$\boxed{\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)}. \tag{13}$$

На каждой итерации вычисляется градиент и делается шаг в направлении $-\nabla L(\theta_t)$.

Пример. Для $L(u) = u^2$ имеем $L'(u) = 2u$ и

$$u_{t+1} = (1 - 2\eta) u_t.$$

Если $0 < \eta < 1$, итерации сходятся к нулю; при $\eta = 1$ возникает чередование u_t и $-u_t$ без уменьшения модуля; при $\eta > 1$ модуль параметра увеличивается (если $u_0 \neq 0$). Этот пример показывает важность выбора шага, но не является универсальной теоремой сходимости для произвольной L.

Стационарной называется точка, для которой $\nabla L(\theta) = 0$. Стационарная точка может быть минимумом, максимумом или седловой точкой; поэтому остановка градиентного метода сама по себе не гарантирует нахождения минимума.

3.3. Batch, Stochastic и Mini-batch Gradient Descent

Если $L(\theta) = n^{-1} \sum_{i=1}^{n} \ell_i(\theta)$, полный градиент равен

$$g(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta).$$

Его вычисление на большой выборке может быть дорогим. Для mini-batch $B_t \subseteq \{1, \dots, n\}$ используют

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t g_t.$$

Если B_t выбирается случайно равномерным образом и каждая точка имеет одинаковую вероятность попадания в пакет, то при стандартных условиях g_t является несмещённой оценкой полного градиента. При $|B_t| = 1$ говорят о стохастическом градиентном спуске (Stochastic Gradient Descent, SGD), а при $|B_t| = n$ — о полном GD.

3.4. Momentum и Nesterov Momentum

На вытянутой поверхности потерь градиентный спуск может колебаться поперёк узкой долины и медленно продвигаться вдоль неё. *Momentum* добавляет накопленное направление движения:

$$\begin{aligned} v_{t+1} &= \beta v_t + g_t, \\ \theta_{t+1} &= \theta_t - \eta v_{t+1}, \end{aligned} \qquad 0 \le \beta < 1, \quad v_0 = 0. \tag{14}$$

Предыдущие градиенты влияют на текущий шаг; при однонаправленных градиентах движение ускоряется, а разнонаправленные вклады могут взаимно компенсироваться.

Nesterov Momentum вычисляет градиент не в текущей точке, а в точке предполагаемого смещения по инерции. Одна из стандартных согласованных форм записи имеет вид

$$\begin{aligned} \theta_t^{\text{look}} &= \theta_t - \eta \beta v_t, \\ g_t &= \nabla L(\theta_t^{\text{look}}), \\ v_{t+1} &= \beta v_t + g_t, \\ \theta_{t+1} &= \theta_t - \eta v_{t+1}. \end{aligned} \tag{15}$$

В Momentum градиент берётся в θ_t; в методе Нестерова место его вычисления сдвинуто с учётом предыдущего движения. Если $\beta = 0$, обе схемы переходят к обычному градиентному спуску.

3.5. AdaGrad и RMSProp

Изменения разных параметров могут происходить в разных масштабах. *AdaGrad* адаптирует шаг по каждой координате:

$$\begin{aligned} G_{t+1} &= G_t + g_t \odot g_t, \qquad G_0 = 0, \\ \theta_{t+1} &= \theta_t - \eta \, \frac{g_t}{\sqrt{G_{t+1}} + \varepsilon}. \end{aligned} \tag{16}$$

Здесь $\odot$ обозначает покоординатное произведение, а квадратный корень, сложение с $\varepsilon > 0$ и деление в формуле обновления выполняются покоординатно. Координаты, по которым ранее наблюдались большие градиенты, получают меньший эффективный шаг. Поскольку G_t только растёт, размер шага со временем может стать слишком малым.

RMSProp вместо суммы всей истории использует экспоненциально сглаженный квадрат градиента:

$$\begin{aligned} s_{t+1} &= \rho s_t + (1-\rho)(g_t \odot g_t), \quad s_0 = 0, \\ \theta_{t+1} &= \theta_t - \eta \frac{g_t}{\sqrt{s_{t+1}} + \varepsilon}, \end{aligned} \qquad 0 \le \rho < 1. \tag{17}$$

За счёт убывающих весов давние градиенты постепенно теряют влияние.

3.6. Adam

Adam объединяет сглаживание самого градиента (идея Momentum) и сглаживание его квадрата (идея RMSProp). При $m_0 = s_0 = 0$ и $g_t = \nabla L(\theta_t)$ или его mini-batch-оценке:

$$\begin{aligned} m_{t+1} &= \beta_1 m_t + (1-\beta_1) g_t, \\ s_{t+1} &= \beta_2 s_t + (1-\beta_2)(g_t \odot g_t), \\ \widehat{m}_{t+1} &= \frac{m_{t+1}}{1-\beta_1^{t+1}}, \\ \widehat{s}_{t+1} &= \frac{s_{t+1}}{1-\beta_2^{t+1}}, \\ \theta_{t+1} &= \theta_t - \eta \frac{\widehat{m}_{t+1}}{\sqrt{\widehat{s}_{t+1}} + \varepsilon}. \end{aligned} \tag{18}$$

Множители $1-\beta_1^{t+1}$ и $1-\beta_2^{t+1}$ вводятся для коррекции смещения, связанного с нулевой инициализацией скользящих средних. Все квадратные корни и деления в векторных формулах Adam выполняются покоординатно, как и в AdaGrad и RMSProp.

Разные оптимизаторы по-разному используют информацию о прошлых градиентах и по-разному выбирают длины шагов. Ни один из них не гарантирует лучшего результата на любой задаче. При сравнении важно учитывать не только число итераций, но и время вычислений, число обращений к градиенту, начальные условия и настройки метода [6].

4. Решающие деревья и ансамбли

4.1. Структура решающего дерева

В отличие от линейной модели решающее дерево (Decision Tree) разбивает пространство признаков на области и ставит каждой конечной области в соответствие предсказание. Внутренние вершины дерева содержат условия на признаки, а листья — итоговые ответы.

Для числового признака типичный вопрос имеет вид

$$x_j \le t.$$

Пусть S — множество индексов объектов, дошедших до некоторой вершины. Для пары (j, t) оно разбивается на

$$S_L(j,t) = \{i \in S : x_{ij} \le t\}, \qquad S_R(j,t) = S \setminus S_L(j,t).$$

В регрессии лист обычно предсказывает среднее значение ответов попавших в него объектов. В классификации — наиболее частый класс или оценку распределения классов. Таким образом, дерево строит кусочно-постоянную по признакам функцию.

4.2. Критерий качества разбиения

Чтобы выбрать признак и порог, вводится *неоднородность* $H(S)$ множества объектов. Разбиение считается полезным, если в потомках ответы становятся более однородными. Один из стандартных критериев качества имеет вид

$$Q(S,j,t) = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R). \tag{19}$$

Здесь $|S|$ — число объектов в вершине. Алгоритм выбирает

$$(j^*, t^*) \in \arg\max_{j,t} Q(S,j,t).$$

Для регрессии можно использовать среднеквадратичное отклонение от среднего в вершине:

$$H_{\text{reg}}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \overline{y}_S)^2, \qquad \overline{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i.$$

Для классификации, если $p_k(S)$ — доля объектов класса k в вершине, обычно применяют *индекс Джини*

$$H_{\text{Gini}}(S) = 1 - \sum_{k=1}^{K} p_k(S)^2 \tag{20}$$

или *энтропию*

$$H_{\text{ent}}(S) = -\sum_{k=1}^{K} p_k(S) \log p_k(S), \tag{21}$$

где полагают $0 \log 0 = 0$. Оба критерия равны нулю, когда все объекты вершины принадлежат одному классу, и увеличиваются при смешивании классов. Для двух классов с долями p и $1-p$ индекс Джини равен $2p(1-p)$ и максимален при $p = 1/2$.

4.3. Жадное построение и дискретная оптимизация

Алгоритм построения дерева рекурсивно выполняет следующие действия:

1) перебирает доступные признаки и допустимые пороги;

2) вычисляет качество $Q(S,j,t)$ каждого разбиения;

3) выбирает пару (j^*, t^*), дающую наибольшее улучшение;

4) разбивает объекты и продолжает обработку потомков;

5) останавливается при выполнении условий остановки (ограничение глубины, малая выборка, недостаточное улучшение).

Метод называют *жадным*: выбор лучшего разбиения делается локально, без гарантии получения глобально оптимальной структуры всего дерева. Число вершин, ветвление и выбор признаков образуют дискретную структуру. Поэтому обычная формула GD непосредственно не описывает процедуру её построения. Это не исключает существования других подходов к оптимизации деревьев, но стандартное обучение дерева не является градиентным спуском по его структуре.

4.4. Random Forest: ансамбль параллельных деревьев

Одно дерево легко подстраивается под особенности обучающих данных. *Random Forest* (случайный лес) объединяет много деревьев, обученных на различающихся данных и подмножествах признаков.

Bootstrap — формирование выборки размера n путём случайного выбора объектов исходной выборки *с возвращением*. Из-за возвращения некоторые объекты повторяются, другие не попадают в выбранный набор. Для каждого дерева часто используют отдельную bootstrap-выборку; при поиске очередного разбиения случайным образом ограничивают набор рассматриваемых признаков.

В задаче регрессии ответы деревьев усредняют:

$$F(x) = \frac{1}{M} \sum_{m=1}^{M} h_m(x).$$

В классификации применяют голосование деревьев или усреднение оценённых ими вероятностей классов. Главная роль случайности — уменьшить сходство ошибок отдельных деревьев; усреднение помогает снизить разброс итоговой модели.

4.5. Gradient Boosting как последовательное улучшение функций

В отличие от леса, *Gradient Boosting* (градиентный бустинг) строит деревья *последовательно*. Пусть F_{m-1} — предсказательная функция после $m-1$ шагов. На следующем шаге строится дерево h_m, исправляющее недостатки текущих предсказаний:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \qquad 0 < \eta \leq 1. \tag{22}$$

Шаг η определяет вклад очередного дерева и аналогичен параметру скорости обучения в градиентном спуске.

Для функции потерь $\ell(y, F(x))$ определим *псевдоостатки* — отрицательные производные по текущим предсказаниям:

$$r_i^{(m)} = - \left. \frac{\partial \ell(y_i, z)}{\partial z} \right|_{z=F_{m-1}(x_i)}. \tag{23}$$

Следующее дерево обучается приближать значения $r_i^{(m)}$ на объектах выборки. При квадратичной потере $\ell(y, z) = \frac{1}{2}(y - z)^2$ получаем

$$r_i^{(m)} = y_i - F_{m-1}(x_i),$$

то есть обычные остатки. Если квадратичная потеря записана без множителя $1/2$, отрицательная производная равна $2(y_i - F_{m-1}(x_i))$: отличие лишь в масштабе.

Термин *градиентный* указывает на выбор поправки по направлению антиградиента функции потерь в пространстве предсказаний. В обычном GD обновляется конечномерный вектор параметров θ; в бустинге последовательно изменяется сама функция F. В практических реализациях могут дополнительно подбираться значения в листьях и применяться регуляризация.

4.6. Связь ансамблей с дальнейшим изложением

Случайный лес объединяет параллельно обученные деревья, а бустинг строит *сумму* постепенно добавляемых функций. Следующая глава посвящена другому способу получать сложные зависимости — *композиции* функций, параметры которых оптимизируются совместно. Так возникает естественный переход от ансамблей к нейронным сетям.

5. Нейронные сети

5.1. Искусственный нейрон и функции активации

Искусственный нейрон получает вектор признаков $x \in \mathbb{R}^d$, вычисляет линейную комбинацию с весами и смещением

$$z = w^T x + b$$

и преобразует её *функцией активации* $\varphi : \mathbb{R} \to \mathbb{R}$:

$$a = \varphi(z) = \varphi(w^T x + b). \tag{24}$$

Величина z называется предактивацией или линейным откликом, а a — выходом нейрона.

Если каждый слой сети использует только линейные или аффинные преобразования, их композиция снова является аффинным преобразованием. Такая глубокая сеть не получит большей выразительной способности, чем одна линейная модель. Нелинейные активации устраняют это ограничение.

Наиболее распространённые примеры:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Для дифференцирования полезны равенства

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \qquad (\tanh z)' = 1 - \tanh^2 z.$$

Производная ReLU равна нулю при $z < 0$ и единице при $z > 0$. В нуле классическая производная отсутствует; при вычислении градиентов на практике выбирают условное значение субградиента, часто равное нулю.

5.2. Многослойный перцептрон как композиция функций

Multilayer Perceptron (MLP, многослойный перцептрон) состоит из последовательности слоёв. Обозначим вход через $a^{(0)} = x$. Для слоя с номером ℓ имеем

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \varphi_\ell\big(z^{(\ell)}\big). \tag{25}$$

Если на предыдущем слое $d_{\ell-1}$ нейронов, а на текущем d_ℓ, то

$$W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad b^{(\ell)}, z^{(\ell)}, a^{(\ell)} \in \mathbb{R}^{d_\ell}.$$

Функция φ_ℓ применяется покоординатно, если не оговорено иное. Все матрицы весов и векторы смещений образуют общий набор обучаемых параметров θ.

Например, функция двухслойной сети записывается как

$$f_\theta(x) = \varphi_2\big(W^{(2)} \varphi_1(W^{(1)} x + b^{(1)}) + b^{(2)}\big)\,.$$

Сеть, в отличие от бустинга, строит *композицию* функций, параметры которой оптимизируются совместно.

Пример прямого прохода. Пусть $x = (1, 2)^T$, $W^{(1)} = I_2$, $b^{(1)} = (0, -1)^T$ и первый слой использует ReLU. Тогда

$$z^{(1)} = (1, 1)^T, \qquad a^{(1)} = (1, 1)^T.$$

При $W^{(2)} = (1, -1)$ и $b^{(2)} = 1$ линейный выход второго слоя равен 1. Этот расчёт показывает, что прямой проход состоит из последовательных матричных операций и вычислений нелинейностей.

5.3. Почему скрытые слои необходимы: задача XOR

Рассмотрим бинарную функцию исключающего ИЛИ (XOR):

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Эти четыре точки нельзя разделить одной прямой на классы 0 и 1. Поэтому одиночный линейный классификатор недостаточен, но композиция нелинейных нейронов может решить задачу. Например, на бинарных входах функция

$$f(x_1, x_2) = \mathrm{ReLU}(x_1 + x_2) - 2\,\mathrm{ReLU}(x_1 + x_2 - 1)$$

принимает значения XOR. Здесь скрытый слой строит два нелинейных признака, а выходной слой складывает их с весами. Это пример того, как нелинейная композиция расширяет класс представимых зависимостей.

5.4. Выход классификационной сети: scores и Softmax

Для задачи классификации на K классов последний слой часто возвращает вектор $z = (z_1, \dots, z_K)^T \in \mathbb{R}^K$. Его координаты называются *scores* или *logits*: они представляют собой необязательно нормированные оценки классов и могут принимать любые вещественные значения.

Определение. Функция Softmax преобразует scores в набор положительных чисел с суммой 1:

$$p_k = \mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K. \tag{26}$$

Вероятностная интерпретация p_k требует предположения о соответствии модели распределению классов; сами значения Softmax не гарантируют статистической калибровки.

Например, при $z = (\log 2, \log 1, \log 1)$ получаем $p = (1/2, 1/4, 1/4)$. Для численной устойчивости при вычислении Softmax обычно вычитают из всех scores одно и то же число, чаще всего $\max_j z_j$: значения p_k при этом не меняются.

Если истинный класс представлен *one-hot*-вектором $y \in \{0, 1\}^K$ с единственной единицей, то *категориальная кросс-энтропия* одного наблюдения равна

$$\ell_{\mathrm{CE}}(y, p) = -\sum_{k=1}^{K} y_k \log p_k. \tag{27}$$

Она равна отрицательному логарифму вероятности, назначенной истинному классу. Для бинарной задачи её аналогом является BCE из главы 2.

5.5. Градиент Softmax и кросс-энтропии

Чтобы обучать классификационную сеть, необходимо найти производную потери по выходным scores. Для Softmax справедливо

$$\frac{\partial p_k}{\partial z_j} = p_k(\mathbf{1}\{k = j\} - p_j).$$

При $\sum_k y_k = 1$ правило цепочки даёт

$$\begin{aligned} \frac{\partial \ell_{\mathrm{CE}}}{\partial z_j} &= -\sum_{k=1}^{K} \frac{y_k}{p_k} \frac{\partial p_k}{\partial z_j} \\ &= -\sum_{k=1}^{K} y_k(\mathbf{1}\{k = j\} - p_j) = p_j - y_j. \end{aligned}$$

В векторной записи

$$\boxed{\nabla_z \ell_{\mathrm{CE}} = p - y}. \tag{28}$$

Это многоклассовый аналог формулы $\partial \ell_{\mathrm{BCE}}/\partial z = p - y$ для логистической регрессии.

5.6. Обучение нейросети и роль Backpropagation

Пусть f_θ — нейронная сеть, а $\ell(y, f_\theta(x))$ — выбранная функция потерь. Как и для линейных моделей, обучающая задача имеет вид

$$\min_\theta L(\theta), \qquad L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(y_i, f_\theta(x_i)\big).$$

Нейросеть может содержать множество слоёв и параметров, поэтому необходимо эффективно вычислять $\nabla_\theta L(\theta)$.

Прямой проход (forward pass) вычисляет все промежуточные активации от входа к выходу. *Обратное распространение* (Backpropagation) вычисляет производные функции потерь по параметрам в обратном направлении, используя правило цепочки. После этого любой подходящий оптимизатор из главы 3 (например, SGD или Adam) обновляет параметры. Таким образом, Backpropagation — *алгоритм вычисления градиента*, а не самостоятельное правило выбора размера шага.

Для mini-batch B вычисляют среднее

$$g_B(\theta) = \frac{1}{|B|}\sum_{i\in B} \nabla_\theta \ell\big(y_i, f_\theta(x_i)\big).$$

Это градиент потерь, усреднённых по выбранному пакету; при случайном равномерном выборе пакета он оценивает градиент по всей обучающей выборке.

5.7. Правило цепочки и матрица Якоби

Для скалярных функций правило цепочки знакомо из анализа:

$$\frac{d}{dx} g(f(x)) = g'(f(x))f'(x).$$

В нейросети каждый слой обычно отображает вектор в другой вектор, поэтому нужен многомерный аналог.

Определение. Если $F : \mathbb{R}^m \to \mathbb{R}^k$ — дифференцируемое отображение, то его *матрица Якоби* имеет вид

$$J_F(x) = \left(\frac{\partial F_i}{\partial x_j}(x)\right)_{\substack{i=1,\dots,k\\ j=1,\dots,m}} \in \mathbb{R}^{k\times m}.$$

Для композиции $G \circ F$ выполняется

$$J_{G\circ F}(x) = J_G(F(x))J_F(x). \tag{29}$$

Если итоговая функция потерь скалярна, а градиенты представлены столбцами, то

$$\nabla_x\big(\ell \circ F\big)(x) = J_F(x)^T \nabla \ell(F(x)).$$

Таким образом, транспонированные матрицы производных позволяют переносить информацию о чувствительности функции потерь от выхода сети к её входу.

5.8. Обратное распространение по слоям

Для одного объекта обозначим его потерю через E и введём *сигнал ошибки слоя*

$$\delta^{(\ell)} = \nabla_{z^{(\ell)}} E. \tag{30}$$

Этот вектор показывает, как изменится итоговая потеря при малом изменении предактиваций $z^{(\ell)}$. По сети назад распространяется *не разность между ответом и предсказанием*, а чувствительность итоговой потери к промежуточным величинам.

Если функции активации применяются покоординатно, для скрытых слоёв правило цепочки даёт

$$\boxed{\delta^{(\ell)} = \big(W^{(\ell+1)}\big)^T \delta^{(\ell+1)} \odot \varphi'_\ell\big(z^{(\ell)}\big)}. \tag{31}$$

Знак $\odot$ обозначает покоординатное произведение векторов. Для последнего слоя $\delta^{(q)}$ вычисляется из функции потерь и выходной активации; например, при Softmax с кросс-энтропией $\delta^{(q)} = p - y$.

Из линейной части слоя $z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$ следуют производные

$$\boxed{\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \big(a^{(\ell-1)}\big)^T}, \qquad \frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}. \tag{32}$$

Например, элемент матрицы весов $W_{ij}^{(\ell)}$ соединяет j-ю активацию предыдущего слоя с i-й предактивацией текущего, поэтому

$$\frac{\partial E}{\partial W_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

После вычисления производных по всем слоям градиенты усредняют по объектам mini-batch и передают оптимизатору.

Небольшой скалярный пример. Пусть $x = 2$, $h = \mathrm{ReLU}(w_1 x)$, $\widehat{y} = w_2 h$, $w_1 = 1$, $w_2 = 3$, $y = 4$, а $E = \frac{1}{2}(\widehat{y} - y)^2$. Тогда $h = 2$, $\widehat{y} = 6$, $E = 2$ и

$$\frac{\partial E}{\partial w_2} = (\widehat{y} - y) h = 4, \qquad \frac{\partial E}{\partial w_1} = (\widehat{y} - y) w_2 x = 12.$$

Здесь производная ReLU равна единице, поскольку $w_1 x > 0$. Пример показывает, как изменение параметра раннего слоя влияет на потерю через параметры последующего слоя.

5.9. Затухание градиента и остаточные связи ResNet

В глубокой сети при многократном применении правила цепочки появляются произведения матриц Якоби соседних слоёв. Если их действие многократно уменьшает длину передаваемых векторов, производные для ранних слоёв становятся очень малы. Это явление называют *затуханием градиента* (vanishing gradient). Возможна и противоположная ситуация — *взрыв градиента*, когда нормы производных становятся чрезмерно большими.

Насыщающиеся активации способны усугублять проблему: например, $0 < \sigma'(z) \leq 1/4$. Однако величина полного градиента зависит также от матриц весов

и других операций, поэтому затухание нельзя объяснять только свойствами одной активации.

Один из способов улучшить обучение глубоких сетей — *остаточная связь* (residual connection). Вместо непосредственного преобразования $F(x)$ остаточный блок вычисляет

$$H(x) = x + F(x). \tag{33}$$

При согласованных размерностях

$$J_H(x) = I + J_F(x),$$

где I — единичная матрица. Путь через слагаемое x позволяет передавать производную через блок даже при слабом вкладе J_F. Это упрощает обучение ряда глубоких архитектур, хотя не даёт универсальной гарантии отсутствия всех проблем с градиентами.

5.10. Свёрточные нейронные сети

Convolutional Neural Network (CNN, свёрточная нейронная сеть) использует локальные фильтры и разделяемые веса. Для двумерного массива A и фильтра K вычисляется

$$B_{u,v} = \sum_{i=0}^{r-1} \sum_{j=0}^{s-1} K_{ij} A_{u+i,v+j}. \tag{34}$$

В стандартных библиотеках глубокого обучения эта операция обычно является *кросс-корреляцией*: коэффициенты фильтра не переворачиваются. В контексте CNN её принято называть свёрткой.

Каждый отклик $B_{u,v}$ — скалярное произведение вектора коэффициентов фильтра с вектором значений локального участка входного массива. Это связывает CNN с линейными моделями главы 2.

Пример. Для участка

$$A = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

результат $\sum_{i,j} K_{ij} A_{ij} = 3$. Этот фильтр реагирует на вертикальную границу между тёмной и светлой областями. На однородном участке из одинаковых значений его отклик равен нулю.

Для нескольких входных каналов и нескольких фильтров каждый выходной канал вычисляется суммированием локальных произведений по всем входным каналам; после этого добавляется смещение и нелинейность.

Преимущество разделяемых весов — уменьшение числа параметров. Для фильтров размера $r \times s$, C_{in} входных и C_{out} выходных каналов число обучаемых параметров слоя с одним смещением на выходной канал равно

$$r s\, C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{35}$$

Оно не зависит напрямую от ширины и высоты изображения, в отличие от полносвязного отображения всех пикселей все выходные элементы.

Наряду со свёрточными слоями архитектуры CNN используют подвыборку (например, max pooling — выбор максимального значения внутри окна), нелинейности и классификационные выходные слои. При наличии функции потерь вся сеть обучается тем же Backpropagation, что и MLP.

6. Муравьиный алгоритм: пример оптимизации без градиентов

6.1. Задача коммивояжёра

В качестве заключительного примера рассмотрим задачу коммивояжёра (Travelling Salesperson Problem, TSP). Пусть имеется N городов, расстояния между которыми заданы положительными числами d_{ij}. Требуется найти замкнутый маршрут, проходящий через каждый город ровно один раз и возвращающийся в начальную вершину, с минимальной суммарной длиной.

Если $\pi = (\pi_1, \dots, \pi_N)$ — перестановка городов, его длина равна

$$C(\pi) = \sum_{k=1}^{N-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_N, \pi_1}. \tag{36}$$

Для фиксированного стартового города существует $(N-1)!$ порядков посещения остальных городов (для симметричных расстояний обратные маршруты порождают одинаковую длину). При росте числа городов перебор всех маршрутов становится вычислительно сложным.

В отличие от задачи обучения гладкой функции потерь пространство возможных маршрутов дискретно, поэтому обычный градиент по координатам перестановки не определён. Один из подходов к приближённому поиску решений — *Ant Colony Optimization* (ACO, муравьиный алгоритм) [10].

6.2. Вероятностный выбор перехода

Каждому доступному ребру (i, j) ставится в соответствие неотрицательная величина τ_{ij} — уровень *феромона*, то есть накопленная информация о том, как часто ребро встречалось в удачных решениях. Также вводится эвристическая привлекательность

$$\nu_{ij} = \frac{1}{d_{ij}}$$

для положительных расстояний.

Если муравей находится в городе i, а A_i — множество ещё не посещённых городов, то вероятность перехода в город $j \in A_i$ задаётся формулой

$$\boxed{P_{ij} = \frac{\tau_{ij}^{\alpha} \nu_{ij}^{\beta}}{\sum\limits_{k \in A_i} \tau_{ik}^{\alpha} \nu_{ik}^{\beta}}}, \tag{37}$$

где $\alpha, \beta \geq 0$ задают относительное влияние феромона и эвристической привлекательности. Формула предполагает положительный знаменатель; на практике для доступных рёбер задают $\tau_{ij} > 0$.

Пример. Пусть доступны два одинаково удалённых города с уровнями феромона $\tau_1 = 2$, $\tau_2 = 1$, а $\alpha = \beta = 1$. Тогда вероятность выбрать первое ребро равна $2/(2+1) = 2/3$, а второе — $1/3$. Таким образом, ребро с большим феромоном предпочтительнее, но выбор остаётся случайным.

6.3. Испарение и обновление феромона

После того как муравьи построили маршруты, уровни феромона обновляются:

$$\boxed{\tau_{ij}^{(t+1)} = (1 - \rho_{\text{evap}})\tau_{ij}^{(t)} + \Delta\tau_{ij}^{(t)}}, \tag{38}$$

где $0 < \rho_{\text{evap}} < 1$ — доля испарения, а $\Delta\tau_{ij}^{(t)}$ — добавка от построенных на итерации t маршрутов.

В одном из распространённых вариантов

$$\Delta\tau_{ij}^{(t)} = \sum_{k:\,(i,j)\in\pi_k} \frac{\kappa}{C(\pi_k)}, \qquad \kappa > 0, \tag{39}$$

где π_k — маршрут k-го муравья. Более короткий маршрут вносит больший вклад в феромон на своих рёбрах. Испарение препятствует бесконечному сохранению старых предпочтений и оставляет возможность перераспределения вероятностей выбора.

6.4. Полный алгоритм и ограничения метода

Общая процедура состоит из следующих шагов:

1) инициализировать уровни феромона положительными числами;

2) для каждого муравья построить маршрут, выбирая переходы по вероятностям (37);

3) вычислить длину каждого маршрута по (36);

4) обновить феромоны по формулам (38)–(39);

5) повторить процесс и сохранить лучший найденный маршрут.

Муравьиный алгоритм сочетает исследование различных вариантов и усиление ранее удачных решений. Он не требует дифференцируемости целевой функции, но в общем случае не гарантирует нахождения глобально кратчайшего маршрута за фиксированное число итераций.

Таким образом, этот метод подчёркивает важное различие: математическая оптимизация может использовать производные, но может опираться и на вероятностный поиск в дискретном множестве допустимых решений.

7. Распределение работы между участниками

Работа над материалами проекта распределена следующим образом:

- **Зайцев Н.** — подготовка и согласование текста отчёта, структуры лекции и презентации;

- **Вейгандт В.** — разработка программных материалов и алгоритмических демонстраций;
- **Козлов В.** — подготовка анимаций, поясняющих динамику изучаемых методов;
- **Горобец О.** — проверка математических выкладок и обоснований;
- **Шагеев Э.** — подготовка статических графиков и иллюстративных материалов для лекции.

Участники совместно обсуждали состав рассматриваемых тем и согласование математических обозначений. Изображения и анимации относятся к сопроводительным материалам доклада; в настоящем текстовом отчёте они не воспроизводятся.

Заключение

В работе рассмотрена общая постановка обучения с учителем как выбора модели по конечной выборке. Показано различие между истинным и эмпирическим риском, сформулирован принцип ERM и объяснено, почему низкая ошибка на обучающих данных сама по себе не гарантирует хорошего качества на новых объектах.

На примере линейной регрессии обучение сведено к минимизации среднеквадратичной ошибки, а на примере логистической регрессии функция потерь получена из вероятностной модели Бернулли и метода максимального правдоподобия. В обоих случаях вычисление градиентов опирается на частные производные и правило цепочки.

Из геометрического смысла градиента получен принцип градиентного спуска и рассмотрены Momentum, Nesterov, AdaGrad, RMSProp и Adam. Эти методы отличаются способом использования текущего и прошлых градиентов, а также покоординатным выбором размера шага.

Решающие деревья иллюстрируют альтернативную постановку, в которой структура модели выбирается дискретным жадным алгоритмом. Для ансамблей показано различие между усреднением деревьев в Random Forest и их последовательным добавлением в Gradient Boosting. Последний метод возвращает идею антиградиента, но применяет её к текущим предсказаниям и строит поправки в виде новых функций.

Наиболее подробно рассмотрены нейронные сети. Показано, что многослойная сеть задаёт композицию линейных преобразований и нелинейных активаций, а её обучение связано с дифференцированием этой композиции. Выведены формулы Softmax с кросс-энтропией, обратного распространения производных и градиентов параметров слоя. Рассмотрены причины затухания градиента, идея остаточных связей ResNet и использование локальных фильтров с общими весами в CNN.

В заключение разобран муравьиный алгоритм на примере задачи коммивояжёра. Его вероятностный выбор переходов, испарение феромона и подкрепление удачных маршрутов показывают, что поиск решений не всегда основывается на дифференцировании целевой функции.

Таким образом, для выбранных методов машинного обучения прослеживается единая математическая логика: *выбор пространства моделей, задание критерия качества и разработка способа поиска подходящего решения.* Математический анализ

особенно явно проявляется при изучении градиентных методов и Backpropagation, где производные становятся непосредственным инструментом построения алгоритма обучения.

Список литературы

[1] Броиловский А. *Лекция 1. Обзор задач машинного обучения.* По мотивам лекции Н. Волкова. ШАД СПб, 2026.

[2] Овчаров С. *Лекция 2. Линейная регрессия и основы оптимизации.* По мотивам лекции Н. Волкова. ШАД СПб, 2026.

[3] Овчаров С. *Лекция 3. Логистическая регрессия и классификация.* По мотивам лекции Н. Волкова. ШАД СПб, 2026.

[4] Школа анализа данных Яндекса. *Учебник по машинному обучению.* https://contest.yandex.ru/tracks/ml/.

[5] Синицин Ф., Соколов Е. *Линейные модели.* Учебник по машинному обучению ШАД Яндекса. https://contest.yandex.ru/tracks/ml/supervised-learning/linear-models.

[6] Zhang A., Lipton Z. C., Li M., Smola A. J. *Dive into Deep Learning.* Разделы об оптимизации, многослойных сетях, свёрточных сетях и остаточных связях. https://d2l.ai/.

[7] Школа анализа данных Яндекса. *Решающие деревья.* https://contest.yandex.ru/tracks/ml/supervised-learning/decision-trees.

[8] Школа анализа данных Яндекса. *Градиентный бустинг.* https://contest.yandex.ru/tracks/ml/supervised-learning/gradient-boosting.

[9] Scikit-learn Developers. *Ensembles: Gradient boosting, random forests, bagging, voting, stacking.* https://scikit-learn.org/stable/modules/ensemble.html.

[10] Dorigo M., Stützle T. *Ant Colony Optimization.* MIT Press, 2004.

[11] *Математический анализ в методах машинного обучения.* Рабочие материалы и лекционная презентация проекта, 2026.

A. Материалы проекта

Электронные материалы проекта, включая исходные тексты и сопровождающие материалы для доклада, размещаются в репозитории:

https://github.com/Z4Y4T/ml_project